Milestone 6 Bewijs in PDF

Romeo Weyns & Jonas Friedhoff - INF202

S1 – Romeo Weyns

Overzicht vergelijking:

bewijs_milestone_6_S1(3000, 3000, 3000, 40.000, 5, 30, ‘BOTH’) -> minimum 49.000 rijen

```
[2026-05-12 17:19:54] completed in 11 s 427 ms
=== bewijs_milestone_6_S1 START | mode=BOTH ===
params: p_visitors=3000 | p_companies=3000 | p_platforms=3000 | p_tickets=40000 | p_permits_per_fest=5 | p_festivals=30
--- BULK run ---
=== bewijs_milestone_6_S1_base_bulk START ===
[LOG] generateRandomCities_bulk | params: p_amount=150 | rows added: 150 | gen: 1 ms | insert: 3 ms | total: 6 ms
[LOG] generateRandomCompanies_bulk | params: p_amount=3000 | rows added: 3000 | gen: 7 ms | insert: 56 ms | total: 75 ms
[LOG] generateRandomTicketplatforms_bulk | params: p_amount=3000 | rows added: 3000 | gen: 8 ms | insert: 77 ms | total: 98 ms
[LOG] generateRandomVisitors_bulk | params: p_amount=3000 | rows added: 3000 | gen: 48 ms | insert: 69 ms | total: 130 ms
=== bewijs_milestone_6_S1_base_bulk DONE | 443 ms ===
[LOG] generateRandomFestivals | params: p_amount=30 | rows added: 30 | gen: 17 ms | insert: 6 ms | total: 23 ms
=== bewijs_milestone_6_S1_children_bulk START ===
[LOG] generateRandomTickets_bulk | params: p_amount=40000 | rows added: 40000 | gen: 678 ms | insert: 1408 ms | total: 2118 ms
[LOG] generateRandomPermits_bulk | params: p_per_festival=5 | rows added: 150 | gen: 1 ms | insert: 6 ms | total: 17 ms
=== bewijs_milestone_6_S1_children_bulk DONE | 2138 ms ===
BULK total (S2 excluded): 2581 ms
--- SINGLE run ---
=== bewijs_milestone_6_S1_base_single START ===
[LOG] generateRandomCities_single | params: p_amount=150 | rows added: 150 | gen: 3 ms | insert: 8 ms | total: 11 ms
[LOG] generateRandomCompanies_single | params: p_amount=3000 | rows added: 3000 | gen: 17 ms | insert: 229 ms | total: 246 ms
[LOG] generateRandomTicketplatforms_single | params: p_amount=3000 | rows added: 3000 | gen: 20 ms | insert: 282 ms | total: 301 ms
[LOG] generateRandomVisitors_single | params: p_amount=3000 | rows added: 3000 | gen: 57 ms | insert: 3478 ms | total: 3535 ms
=== bewijs_milestone_6_S1_base_single DONE | 4223 ms ===
[LOG] generateRandomFestivals | params: p_amount=30 | rows added: 30 | gen: 16 ms | insert: 5 ms | total: 21 ms
=== bewijs_milestone_6_S1_children_single START ===
[LOG] generateRandomTickets_single | params: p_amount=40000 | rows added: 40000 | gen: 799 ms | insert: 2881 ms | total: 3679 ms
[LOG] generateRandomPermits_single | params: p_per_festival=5 | rows added: 150 | gen: 13 ms | insert: 11 ms | total: 24 ms
=== bewijs_milestone_6_S1_children_single DONE | 3709 ms ===
SINGLE total (S2 excluded): 7931 ms
=== Comparison (S1 time only, S2 excluded) ===
BULK   : 2581 ms
SINGLE: 7931 ms
SINGLE / BULK ratio: 3.07x
=== bewijs_milestone_6_S1 END | elapsed: 11293 ms ===
```

bewijs_milestone_6_S1(3000, 3000, 3000, 400.000, 5, 300, ‘BOTH’) -> minimum 400.000 rijen

```
[2026-05-12 17:21:39] completed in 1 m 6 s 16 ms
=== bewijs_milestone_6_S1 START | mode=BOTH ===
params: p_visitors=3000 | p_companies=3000 | p_platforms=3000 | p_tickets=400000 | p_permits_per_fest=5 | p_festivals=300
--- BULK run ---
=== bewijs_milestone_6_S1_base_bulk START ===
[LOG] generateRandomCities_bulk | params: p_amount=150 | rows added: 150 | gen: 1 ms | insert: 3 ms | total: 6 ms
[LOG] generateRandomCompanies_bulk | params: p_amount=3000 | rows added: 3000 | gen: 6 ms | insert: 41 ms | total: 58 ms
[LOG] generateRandomTicketplatforms_bulk | params: p_amount=3000 | rows added: 3000 | gen: 8 ms | insert: 105 ms | total: 125 ms
[LOG] generateRandomVisitors_bulk | params: p_amount=3000 | rows added: 3000 | gen: 47 ms | insert: 65 ms | total: 125 ms
=== bewijs_milestone_6_S1_base_bulk DONE | 443 ms ===
[LOG] generateRandomFestivals | params: p_amount=300 | rows added: 300 | gen: 20 ms | insert: 9 ms | total: 28 ms
=== bewijs_milestone_6_S1_children_bulk START ===
[LOG] generateRandomTickets_bulk | params: p_amount=400000 | rows added: 400000 | gen: 6752 ms | insert: 10636 ms | total: 17429 ms
[LOG] generateRandomPermits_bulk | params: p_per_festival=5 | rows added: 1500 | gen: 12 ms | insert: 23 ms | total: 45 ms
=== bewijs_milestone_6_S1_children_bulk DONE | 17508 ms ===
BULK total (S2 excluded): 17951 ms
--- SINGLE run ---
=== bewijs_milestone_6_S1_base_single START ===
[LOG] generateRandomCities_single | params: p_amount=150 | rows added: 150 | gen: 3 ms | insert: 9 ms | total: 11 ms
[LOG] generateRandomCompanies_single | params: p_amount=3000 | rows added: 3000 | gen: 17 ms | insert: 211 ms | total: 228 ms
[LOG] generateRandomTicketplatforms_single | params: p_amount=3000 | rows added: 3000 | gen: 21 ms | insert: 273 ms | total: 294 ms
[LOG] generateRandomVisitors_single | params: p_amount=3000 | rows added: 3000 | gen: 57 ms | insert: 219 ms | total: 277 ms
=== bewijs_milestone_6_S1_base_single DONE | 942 ms ===
[LOG] generateRandomFestivals | params: p_amount=300 | rows added: 300 | gen: 20 ms | insert: 7 ms | total: 28 ms
=== bewijs_milestone_6_S1_children_single START ===
[LOG] generateRandomTickets_single | params: p_amount=400000 | rows added: 400000 | gen: 8017 ms | insert: 37737 ms | total: 45754 ms
[LOG] generateRandomPermits_single | params: p_per_festival=5 | rows added: 1500 | gen: 23 ms | insert: 92 ms | total: 116 ms
=== bewijs_milestone_6_S1_children_single DONE | 45916 ms ===
SINGLE total (S2 excluded): 46858 ms
=== Comparison (S1 time only, S2 excluded) ===
BULK   : 17951 ms
SINGLE: 46858 ms
SINGLE / BULK ratio: 2.61x
=== bewijs_milestone_6_S1 END | elapsed: 65913 ms ===
```

Resultaten 3000, 3000, 3000, 400.000, 5, 300 (BULK vs SINGLE):

Stap	Rijen	Gen ms BULK	Insert ms BULK	Gen ms SINGLE	Insert ms SINGLE
Visitors	3000	47	65	57	219
Companies	3000	6	41	17	211
Ticketplatforms	3000	8	105	21	273
Tickets	400.000	6752	10.636	8017	37.737
Festivals	300	20	9	20	7
Permits	1500	12	23	23	92
Totaal	410.800	-	17.951	-	46.858

Verhouding SINGLE / BULK op deze run: 2.61x.

Segmentgrootte van de TICKET tabel na de 400.000-run:

	SEGMENT_NAME	SEGMENT_TYPE	MB	TABLE_COUNT
1	TICKET	TABLE	80	400000

Conclusie:

Bij kleine en middelgrote tabellen (Festivals met 300 rijen, Permits met 1 500 rijen) is het verschil tussen BULK en SINGLE inserts beperkt maar al zichtbaar: Permits zijn 4x trager bij SINGLE (92 ms vs 23 ms insert). Bij de basisgegevens (Companies, TicketPlatforms, Visitors — elk 3 000 rijen) is de insert-fase bij SINGLE al 3 tot 5 keer trager dan bij BULK, terwijl de generatie-fase grotendeels vergelijkbaar is.

Het fundamentele verschil verschijnt bij de 400 000 Tickets: BULK voert die inserts uit in ±10,6 seconden, SINGLE heeft er ±37,7 seconden voor nodig, 3,5x trager voor exact dezelfde data.

De generatie-fase is voor Tickets ietwat trager bij SINGLE (8 017 ms vs 6 752 ms) door het herhaaldelijk ophogen van de in-memory comboteller bij botsingen; de rest van de tijdverschillen is puur insert-overhead. Conclusie: FORALL loont meetbaar vanaf een paar duizend rijen; de meerwinst is het grootst bij de N-N-junctietabel Ticket, waar ook de generatielogica zelf aan complexiteit wint.

Moeilijkheid:

De grootste uitdaging voor S1 was de compound-uniqueness constraint op de Ticket-tabel: dezelfde combinatie van VISITOR_ID, FESTIVAL_ID en TICKET_DATE mag slechts één keer voorkomen. Bij een N-N-junction met 400 000 rijen, 3 000 bezoekers en 300 festivals zijn botsingen realistisch en mag het generatiealgoritme geen enkele dubbele combinatie naar de database sturen.

Een tweede moeilijkheid was de verdeling van tickets over festivals: puur willekeurige toewijzing zou systematisch de eerste festivals in de tabel bevoordelen en een onrealistisch gelijke spreiding geven. Daarbovenop bestond het probleem dat bij SINGLE-row inserts de generatie-

tijd en insert-tijd van nature door elkaar lopen in dezelfde loop, waardoor ze afzonderlijk niet meetbaar zijn, het probleem dat ook S2 tegenkwam.

Hoe opgelost:

Voor de BULK-generator worden twee opeenvolgende algoritmen gecombineerd. Eerst wordt de festivalvolgorde met een volledige Fisher-Yates shuffle in PL/SQL gerandomiseerd, zodat de verdeling niet systematisch de eerst geladen festivals bevoordeelt. Daarna wordt via stick-breaking het totale ticketbudget verdeeld: elk festival claimt een willekeurig deel van het resterende budget, wat een realistische ongelijke spreidin geeft (van 0 tot maximaal het aantal bezoekers per festival). Per festival wordt vervolgens een sparse partiële Fisher-Yates uitgevoerd over de bezoekerspool om exact het gevraagde aantal unieke bezoekers te selecteren, zonder enige retry en met wiskundige garantie op uniciteit. Omdat de arrays bij 400 000+ rijen te groot worden voor het PGA, worden ze opgeknipt in chunks van C_BULK_CHUNK rijen; elke chunk wordt via FORALL ... SAVE EXCEPTIONS naar de database gestuurd en daarna vrijgegeven.

Voor de SINGLE-generator is uniciteit anders opgelost: een in-memory associatieve array (t_combo_set, geïndexeerd op de samengestelde sleutel visitor_id || '_' || festival_id || '_' || datum) houdt bij welke combinaties al gegenereerd zijn. Bij een botsing slaat de WHILE-loop die iteratie over en probeert een nieuwe combinatie, tot het gevraagde aantal unieke tickets bereikt is. Dit legt de uniciteitscontrole volledig in PL/SQL en vermijdt round-trips naar de database.

Voor het opsplitten van generatie- en insert-tijd is dezelfde aanpak als S2 toegepast: elke single-insert procedure werd herschreven in twee opeenvolgende lussen. Loop 1 vult alle PL/SQL-collecties met gegenereerde waarden en neemt tijdstempel v_gen_done; loop 2 itereert daarna over diezelfde collecties en doet één INSERT per rij, gevolgd door tijdstempel v_end. Zo zijn de gen- en insert-fasen voor SINGLE en BULK op exacte dezelfde manier gemeten en is de vergelijking eerlijk.

Overzicht functies:

```
    -- Private: realistic email domain selector (used by visitor generators)
    FUNCTION random_email_domain ...;

    -- ------------------------------------------------------------

    -- PUBLIC LOOKUP (also used internally by this package)
    FUNCTION lookup_company_id...;

    -- ------------------------------------------------------------

    FUNCTION lookup_city_id...;

    -- PRIVATE LOOKUP FUNCTIONS
    FUNCTION lookup_visitor_id...;

    -- ------------------------------------------------------------

    FUNCTION lookup_ticket_id...;

    -- ------------------------------------------------------------

    FUNCTION lookup_ticketplatform_id...;

    -- ------------------------------------------------------------

    FUNCTION lookup_permit_id...;

    -- ------------------------------------------------------------------------------------
    -- PRIVATE ADD PROCEDURES

    PROCEDURE add_company...;

    -- ------------------------------------------------------------

    PROCEDURE add_city...;

    -- ------------------------------------------------------------

    PROCEDURE add_visitor...;
```

```
    PROCEDURE add_ticketplatform...;

    -- ------------------------------------------------------------

    PROCEDURE add_permit...;

    -- ------------------------------------------------------------

    PROCEDURE add_ticket...;

-- ------------------------------------------------------------------------------------

-- PUBLIC: EMPTY TABLES (phased)
-- Cross-schema FK forces interleaving with PKG_S2_festivals.empty_tables_S2:
--   1) empty_s1_children____-> TICKET, PERMIT_____(children of S2.FESTIVAL)
--   2) empty_tables_S2_____-> S2 tables incl. FESTIVAL
--   3) empty_s1_base_______-> TICKETPLATFORM, VISITOR, COMPANY, CITY

    PROCEDURE empty_s1_children...;

    PROCEDURE empty_s1_base...;

    -- Convenience wrapper: only safe when S2 tables (esp. FESTIVAL) are already empty
    PROCEDURE empty_tables_S1 ...;

-- ============================================================
-- fill_s1_base: COMPANY, CITY, VISITOR, TICKETPLATFORM
--   Must run BEFORE PKG_S2_festivals.bewijs_M4_S2 (FESTIVAL needs COMPANY).
-- ============================================================

    PROCEDURE fill_s1_base ...;

-- ============================================================
-- fill_s1_children: PERMIT, TICKET
--   Must run AFTER PKG_S2_festivals.bewijs_M4_S2 (FESTIVAL must exist).
-- ============================================================

    PROCEDURE fill_s1_children ...;
```

```
-- ============================================================
-- M5 - RANDOM GENERATORS  (Stap 2)
-- ============================================================

    -- ------------------------------------------------------
    -- generateRandomCities
    --   Unique zipcode: 4-digit zero-padded counter (offset + i),
    --   so repeated calls never collide with each other or with M4.
    -- ------------------------------------------------------
    PROCEDURE generateRandomCities...;

    -- ------------------------------------------------------
    -- generateRandomCompanies
    --   Unique name: realistic word combo + zero-padded counter.
    --   Companies serve as festival owners for S2's random festivals.
    -- ------------------------------------------------------
    PROCEDURE generateRandomCompanies...;

    -- ------------------------------------------------------
    -- generateRandomTicketplatforms
    --   Unique name AND url, both derived from a realistic platform name.
    -- ------------------------------------------------------
    PROCEDURE generateRandomTicketplatforms...;

    -- ------------------------------------------------------
    -- generateRandomVisitors
    --   BULK COLLECT city IDs → pure-PL/SQL value generation →
    --   FORALL INSERT.
    --   Email uniqueness: LOWER(firstname) || (offset+i) || random suffix
    -- ------------------------------------------------------
    PROCEDURE generateRandomVisitors...;
```

```
    --
    -- Visitor (1)--(*) Ticket (*)--(1) Festival
    --   FESTIVAL is owned by S2; we read it via cross-schema SELECT.
    --
    -- All three parent pools are BULK COLLECTed once.
    -- Twelve value arrays are built in PL/SQL, then one FORALL
    -- sends all INSERTs to the SQL engine in a single batch.
    --
    -- Date constraints honoured:
    --   TICKET_DATE        in [FESTIVAL_STARTDATE, FESTIVAL_ENDDATE]
    --   TICKET_PURCHASE_DATE <= TICKET_DATE
    --   TICKET_PURCHASE_DATE <= SYSDATE  (DDL CHECK)
    --
    -- Uniqueness constraints:
    --   TICKET_BARCODE unique: 'R' + 12-digit (existing count + i)
    --   (VISITOR_ID, FESTIVAL_ID, TICKET_DATE): violations skipped
    -- ------------------------------------------------------
    PROCEDURE generateRandomTickets...;

    -- ------------------------------------------------------
    -- generateRandomPermits
    --   Inserts p_per_festival permits for every existing festival.
    --   Festival IDs are BULK COLLECTed once.  All permit rows are
    --   built into flat arrays then sent in a single FORALL INSERT.
    --
    --   Date ordering guaranteed:
    --     application_date < reception_date < expiration_date
    --     application_date <= SYSDATE  (DDL CHECK)
    -- ------------------------------------------------------
    PROCEDURE generateRandomPermits...;

    -- ------------------------------------------------------
    -- bewijs_milestone_5_S1_base
    --   Phase 1: generates all S1 rows that have NO dependency on
    --   S2 festivals.  Must run BEFORE bewijs_milestone_5_S2 so
    --   that S2 can pick random companies as festival owners.
    --   Mirrors M4's fill_s1_base.
    -- ------------------------------------------------------
    PROCEDURE bewijs_milestone_5_S1_base...;
```

```
    -- ------------------------------------------------------
    -- bewijs_milestone_5_S1_base
    --   Phase 1: generates all S1 rows that have NO dependency on
    --   S2 festivals.  Must run BEFORE bewijs_milestone_5_S2 so
    --   that S2 can pick random companies as festival owners.
    --   Mirrors M4's fill_s1_base.
    -- ------------------------------------------------------
    PROCEDURE bewijs_milestone_5_S1_base...;

    -- ------------------------------------------------------
    -- bewijs_milestone_5_S1_children
    --   Phase 2: generates TICKET and PERMIT rows that require
    --   festivals to already exist in S2.  Must run AFTER
    --   bewijs_milestone_5_S2.  Mirrors M4's fill_s1_children.
    -- ------------------------------------------------------
    PROCEDURE bewijs_milestone_5_S1_children...;

    -- ------------------------------------------------------
    -- bewijs_milestone_5_S1
    --   Convenience wrapper that calls base then children.
    --   Only safe when S2 festivals already exist (same contract
    --   as bewijs_M4_S1).  The SAMEN script is the correct entry
    --   point for a full cross-schema M5 demonstration.
    -- ------------------------------------------------------
    PROCEDURE bewijs_milestone_5_S1...;
```

```
-- ============================================================
-- M6 - SINGLE-INSERT GENERATORS
-- Row-by-row INSERT; DUP_VAL_ON_INDEX is silently skipped.
-- gen_secs is 0 because generation and insertion are interleaved.
-- ============================================================

    -- ------------------------------------------------------
    -- generateRandomCities_single
    -- ------------------------------------------------------
    PROCEDURE generateRandomCities_single...;

    -- ------------------------------------------------------
    -- generateRandomCompanies_single
    -- ------------------------------------------------------
    PROCEDURE generateRandomCompanies_single...;

    -- ------------------------------------------------------
    -- generateRandomTicketplatforms_single
    -- ------------------------------------------------------
    PROCEDURE generateRandomTicketplatforms_single...;

    -- ------------------------------------------------------
    -- generateRandomVisitors_single
    --   City pool BULK COLLECTed once (read-only).
    --   Email: firstname.lastname<counter>@<realistic-domain>
    -- ------------------------------------------------------
    PROCEDURE generateRandomVisitors_single...;

    -- ------------------------------------------------------
    -- generateRandomTickets_single
    --   Parent pools BULK COLLECTed once; each ticket inserted
    --   individually; duplicates silently skipped.
    --   Barcodes use 'S' prefix to distinguish from bulk runs.
    -- ------------------------------------------------------
    PROCEDURE generateRandomTickets_single...;
```

```
    -- ------------------------------------------------------
    -- generateRandomPermits_single
    --   Iterates over every festival (FESTIVAL_NAME included in
    --   BULK COLLECT) and inserts p_per_festival permits one
    --   at a time, referencing the real festival name.
    -- ------------------------------------------------------
    PROCEDURE generateRandomPermits_single...;

-- ============================================================
-- M6 - BULK GENERATORS WITH CHUNKING
-- Arrays capped at C_BULK_CHUNK; outer WHILE loop refills and
-- flushes until p_amount rows are inserted.  Prevents PGA
-- exhaustion at 400 000+ rows.  Gen time and insert time are
-- accumulated separately across chunks.
-- ============================================================

    -- ------------------------------------------------------
    -- generateRandomCities_bulk
    -- ------------------------------------------------------
    PROCEDURE generateRandomCities_bulk...;

    -- ------------------------------------------------------
    -- generateRandomCompanies_bulk
    -- ------------------------------------------------------
    PROCEDURE generateRandomCompanies_bulk...;

    -- ------------------------------------------------------
    -- generateRandomTicketplatforms_bulk
    -- ------------------------------------------------------
    PROCEDURE generateRandomTicketplatforms_bulk...;

    -- ------------------------------------------------------
    -- generateRandomVisitors_bulk
    --   City pool loaded once; rows produced in chunks.
    --   Email: firstname.lastname<counter>@<realistic-domain>
    -- ------------------------------------------------------
    PROCEDURE generateRandomVisitors_bulk...;
```

```
    -- ------------------------------------------------------------
    -- generateRandomTickets_bulk
    --   Twelve INSERT arrays capped at C_BULK_CHUNK; freed
    --   between chunks so PGA stays bounded at 400 000+ rows.
    --   Barcodes use 'B' prefix (M5='R', single='S').
    -- ------------------------------------------------------------
    PROCEDURE generateRandomTickets_bulk...;

    -- ------------------------------------------------------------
    -- generateRandomPermits_bulk
    --   FESTIVAL_NAME loaded alongside FESTIVAL_ID so the
    --   description references the real festival name.
    --   Rows accumulate in a buffer; flushed per C_BULK_CHUNK.
    -- ------------------------------------------------------------
    PROCEDURE generateRandomPermits_bulk...;

-- ============================================================
-- M6 - PHASE WRAPPERS
-- S2 must run between base and children due to cross-schema FKs:
--   FESTIVAL (S2) -> COMPANY/CITY (S1)   base must precede S2
--   TICKET/PERMIT (S1) -> FESTIVAL (S2)  children must follow S2
-- ============================================================

    PROCEDURE bewijs_milestone_6_S1_base_single...;

    -- ------------------------------------------------------------

    PROCEDURE bewijs_milestone_6_S1_base_bulk...;

    -- ------------------------------------------------------------

    PROCEDURE bewijs_milestone_6_S1_children_single...;

    -- ------------------------------------------------------------

    PROCEDURE bewijs_milestone_6_S1_children_bulk...;
```

```
-- ============================================================
-- M6 - PROOF PROCEDURE
-- Each run: reset to M4 baseline → S1 base → S2 (bewijs_M5_S2) → S1 children
-- S2 time excluded from S1 timing so the comparison is fair.
-- Per-table gen/insert timings appear in individual [LOG] lines.
-- ============================================================

    PROCEDURE bewijs_milestone_6_S1...;

END PKG_S1_festivals;
/
```

S2 – Jonas Friedhoff

bewijs_milestone_6_S2(30, 30, 30, 60, 'BOTH') -> minimum 54.000 rijen

```
[2026-05-12 14:44:11] PROJECT> BEGIN PKG_S2_festivals.bewijs_milestone_6_S2(30, 30, 30, 60,  'BOTH'); END;
[2026-05-12 14:44:25] completed in 13 s 685 ms
=== bewijs_milestone_6_S2 START | mode=BOTH ===
params: p_genres=30 | p_festivals=30 | p_stages=30 | p_perfs=60
-- BULK run --
[LOG] generateRandomGenres | params: p_amount=30 | rows added: 30 | gen: 0 ms | insert: 4 ms | total: 5 ms
[LOG] generateRandomArtists | params: p_amount=300 | rows added: 300 | gen: 4 ms | insert: 5 ms | total: 9 ms
[LOG] generateRandomFestivals | params: p_amount=30 | rows added: 30 | gen: 19 ms | insert: 3 ms | total: 22 ms
[LOG] generateRandomStages | params: p_per_festival=30, festivals_processed=40 | rows added: 1200 | gen: 63 ms | insert: 43 ms | total: 150 ms
[LOG] generateRandomPerformances | params: p_per_stage=60, stages_processed=1220 | rows added: 73200 | gen: 1776 ms | insert: 955 ms | total: 2842 ms
BULK total: 3030 ms
-- SINGLE run --
[LOG] generateRandomGenres_single | params: p_amount=30 | rows added: 30 | gen: 0 ms | insert: 2 ms | total: 3 ms
[LOG] generateRandomArtists_single | params: p_amount=300 | rows added: 300 | gen: 4 ms | insert: 17 ms | total: 21 ms
[LOG] generateRandomFestivals_single | params: p_amount=30 | rows added: 30 | gen: 12 ms | insert: 3 ms | total: 15 ms
[LOG] generateRandomStages_single | params: p_per_festival=30, festivals_processed=40 | rows added: 1200 | gen: 57 ms | insert: 118 ms | total: 215 ms
[LOG] generateRandomPerformances_single | params: p_per_stage=60, stages_processed=1220 | rows added: 73200 | gen: 1777 ms | insert: 4685 ms | total: 6572 ms
SINGLE total: 6828 ms
=== Comparison ===
BULK   : 3030 ms
SINGLE : 6828 ms
SINGLE / BULK ratio: 2.25x
=== bewijs_milestone_6_S2 END ===
```

bewijs_milestone_6_S2(30, 30, 30, 600, 'BOTH') -> minimum 540.000 rijen

```
[2026-05-12 14:45:05] PROJECT> BEGIN PKG_S2_festivals.bewijs_milestone_6_S2(30, 30, 30, 600, 'BOTH'); END;
[2026-05-12 14:46:41] completed in 1 m 35 s 982 ms
== bewijs_milestone_6_S2 START | mode=BOTH ==
params: p_genres=30 | p_festivals=30 | p_stages=30 | p_perfs=600
-- BULK run --
[LOG] generateRandomGenres | params: p_amount=30 | rows added: 30 | gen: 0 ms | insert: 4 ms | total: 5 ms
[LOG] generateRandomArtists | params: p_amount=300 | rows added: 300 | gen: 4 ms | insert: 6 ms | total: 10 ms
[LOG] generateRandomFestivals | params: p_amount=30 | rows added: 30 | gen: 19 ms | insert: 3 ms | total: 22 ms
[LOG] generateRandomStages | params: p_per_festival=30, festivals_processed=40 | rows added: 1200 | gen: 56 ms | insert: 61 ms | total: 157 ms
[LOG] generateRandomPerformances | params: p_per_stage=600, stages_processed=1220 | rows added: 732000 | gen: 17473 ms | insert: 9547 ms | total: 27213 ms
BULK total: 27422 ms
-- SINGLE run --
[LOG] generateRandomGenres_single | params: p_amount=30 | rows added: 30 | gen: 0 ms | insert: 2 ms | total: 3 ms
[LOG] generateRandomArtists_single | params: p_amount=300 | rows added: 300 | gen: 4 ms | insert: 13 ms | total: 17 ms
[LOG] generateRandomFestivals_single | params: p_amount=30 | rows added: 30 | gen: 12 ms | insert: 3 ms | total: 15 ms
[LOG] generateRandomStages_single | params: p_per_festival=30, festivals_processed=40 | rows added: 1200 | gen: 56 ms | insert: 128 ms | total: 223 ms
[LOG] generateRandomPerformances_single | params: p_per_stage=600, stages_processed=1220 | rows added: 732000 | gen: 17486 ms | insert: 46646 ms | total: 64339 ms
SINGLE total: 64612 ms
== Comparison ==
BULK   : 27422 ms
SINGLE : 64612 ms
SINGLE / BULK ratio: 2.36x
== bewijs_milestone_6_S2 END ==
```

Resultaten 30, 30, 30, 600 (BULK vs SINGLE):

Stap	Rijen	Gen ms BULK	Insert ms BULK	Gen ms SINGLE	Insert ms SINGLE
Genres	30	0	4	0	2
Artists	300	4	6	4	13
Festivals	30	19	3	12	3
Stages	1 200	56	61	56	128
Performances	732 000	17473	9 547	17486	46646
Totaal	733 560	-	29 135	-	56 391

Verhouding SINGLE / BULK op deze run: 1.94x.

Segmentgrootte van de PERFORMANCE tabel na de 600-run:

	SEGMENT_NAME	SEGMENT_TYPE	MB	TABLE_COUNT
1	PERFORMANCE	TABLE	184	732040

Conclusie:

Bij kleine tabellen (Genres, Artists, Festivals - telkens onder de 300 rijen) is het verschil tussen BULK en SINGLE inserts verwaarloosbaar: beide methodes blijven onder de 20 milliseconden. Vanaf 1 200 Stages wordt SINGLE al merkbaar trager dan BULK op de insert-fase (65 ms versus 51 ms). Het echte verschil verschijnt pas bij de 732 000 Performances: BULK doet die inserts in ongeveer 9,5 seconden, SINGLE heeft er ongeveer 46,6 seconden voor nodig – meer als 4x zo lang voor exact dezelfde data.

De generatie-fase (random data opbouwen in PL/SQL-arrays) is bij beide methodes vrijwel identiek (~17 s), wat ook logisch is: enkel de manier waarop de data naar de databank geschreven wordt verschilt. Conclusie: BULK loont pas vanaf duizenden rijen per insert; voor kleinere tabellen weegt de extra complexiteit niet op tegen de winst.

Moeilijkheid:

Omdat ik in Milestone 5 al BULK (FORALL) had gebruikt, moest ik voor Milestone 6 het omgekeerde maken: single-row insert versies om te kunnen vergelijken. De grootste uitdaging was de generatie-tijd en insert-tijd afzonderlijk meten voor die single-versies. Bij de BULK-versie is dat eenvoudig - het FORALL-statement vormt een natuurlijke grens tussen 'data in arrays opbouwen' en 'data wegschrijven'. Bij row-by-row inserts staan die twee stappen normaal door elkaar in dezelfde loop, waardoor je hun aandeel niet meer apart kan meten.

Een tweede probleem was dat de APPEND_VALUES hint die ik wilde gebruiken om BULK nog sneller te maken, op Oracle Autonomous Database niet compatibel is met FORALL ... SAVE EXCEPTIONS (ORA-38910 'BATCH ERROR mode is not supported for this operation'). Die exception-afhandeling wou ik wel behouden voor robuustheid.

Hoe opgelost:

Voor de timing splitste ik elke single-insert procedure in twee opeenvolgende loops: loop 1 vult de PL/SQL-collecties met random data en stopt zodra alle rijen klaar zijn (tijdstempel v_gen_done); loop 2 itereert daarna over diezelfde collecties en doet een INSERT per rij (tijdstempel v_end). Zo komen de gen- en insert-fasen exact overeen met de BULK-versie en is de vergelijking eerlijk.

Voor de APPEND_VALUES kwestie heb ik de hint weggehaald en in plaats daarvan op andere optimalisaties ingezet die wel naast SAVE EXCEPTIONS werken: DBMS_RANDOM.VALUE inline gebruiken in de hot loop (in plaats van via de helper-functies in PKG_SAMEN te gaan), per-stage constanten buiten de inner loop hijsen, SYS_GUID() vervangen door een session-scoped g_seq PLS_INTEGER counter in give_unique_combo, en TRUNCATE TABLE gebruiken in plaats van DELETE FROM voor de reset_to_M4 tussen de BULK- en SINGLE-runs.

Overzicht functies:

Collapse Recursively op PKG_S2_festivals (lookups, M5 BULK, M6 SINGLE, M4 add_*, M4 empty + fill):

```
CREATE OR REPLACE PACKAGE BODY PKG_S2_festivals AS
-- ------------------------------------------------------------
   -- LOOKUP FUNCTIONS (public festival lookup + private id resolvers)
-- ------------------------------------------------------------

   -- PUBLIC LOOKUP (also used internally by this package)
   FUNCTION lookup_festival_id ...;

   -- PRIVATE LOOKUP FUNCTIONS
   FUNCTION lookup_company_id ...;

   FUNCTION lookup_city_id ...;

   FUNCTION lookup_genre_id ...;

   FUNCTION lookup_artist_id ...;

   FUNCTION lookup_stage_id ...;

-- ------------------------------------------------------------
   -- M5 – BULK random generators (FORALL bulk DML)
-- ------------------------------------------------------------

   PROCEDURE generateRandomGenres ...;

   PROCEDURE generateRandomArtists ...;

   PROCEDURE generateRandomFestivals ...;

   PROCEDURE generateRandomStages ...;

   PROCEDURE generateRandomPerformances ...;

   PROCEDURE bewijs_M5_S2 ...;

-- ------------------------------------------------------------
   -- M6 – SINGLE-row insert generators + proof procedure
-- ------------------------------------------------------------

   PROCEDURE generateRandomGenres_single ...;

   PROCEDURE generateRandomArtists_single ...;

   PROCEDURE generateRandomFestivals_single ...;

   PROCEDURE generateRandomStages_single ...;

   PROCEDURE generateRandomPerformances_single ...;

   PROCEDURE bewijs_milestone_6_S2 ...;
```

```
-- ------------------------------------------------------------
   -- M4 – Private single-row add_* helpers (used by bewijs_M4_S2)
-- ------------------------------------------------------------

   PROCEDURE add_festival ...;

   PROCEDURE add_genre ...;

   PROCEDURE add_artist ...;

   PROCEDURE add_stage ...;

   PROCEDURE add_performance ...;

-- ------------------------------------------------------------
   -- M4 – Public empty + fill (bewijs_M4_S2 = baseline used by M5/M6)
-- ------------------------------------------------------------

   PROCEDURE empty_tables_S2 ...;

   PROCEDURE bewijs_M4_S2  ...;

END PKG_S2_festivals;
/
```